Data-Structures & Algorithms

with

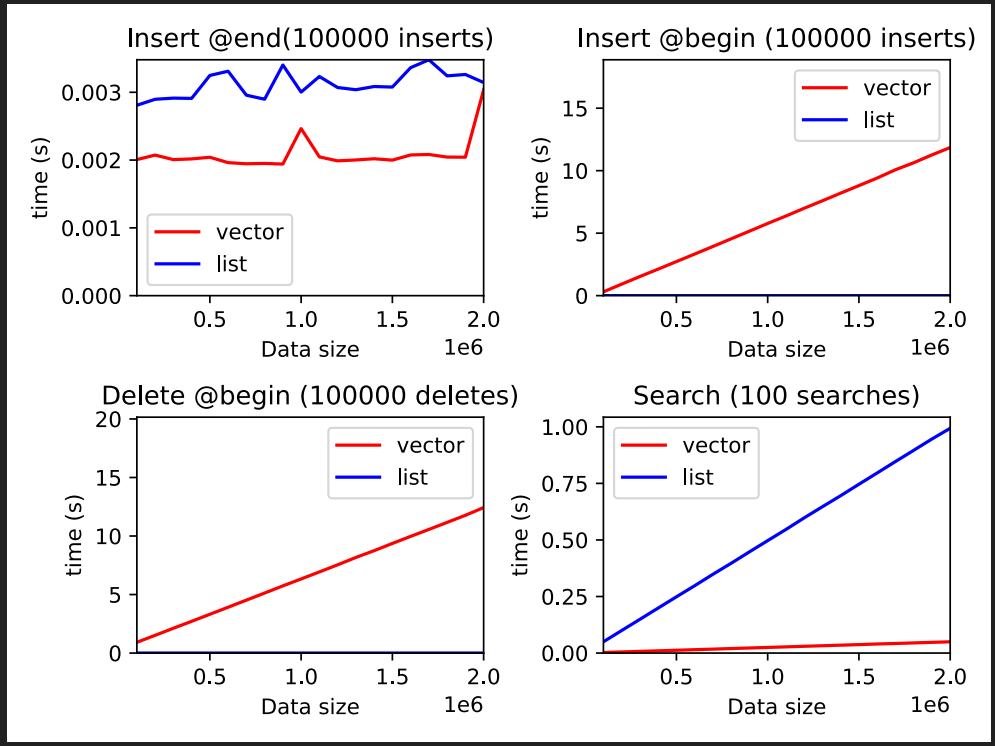
Manoj Prabhakaran

Lecture 4

Linked Lists: Doubly-Linked Lists

Recap: Lists vs. Arrays

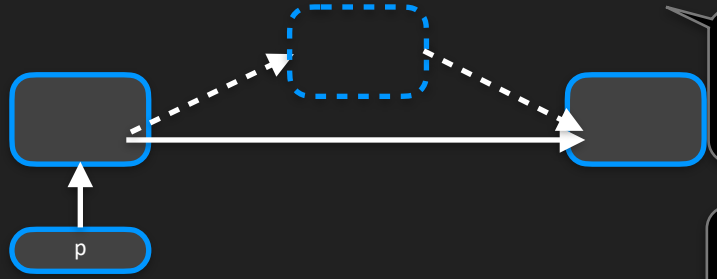
- Two simple data structures with Θ(n) searches
  - Concretely, searches in an array are faster (e.g., by an order of magnitude)
- Lists allow Θ(1) insertion and deletion anywhere in the list, including the beginning, but it is Θ(n) for vectors
  - Aside: insert/delete in vector is much faster than search, due to hardware support of bulk moves



O(1) Insertion and Deletion in a List

- Given a pointer to the node after which the insertion/deletion should be

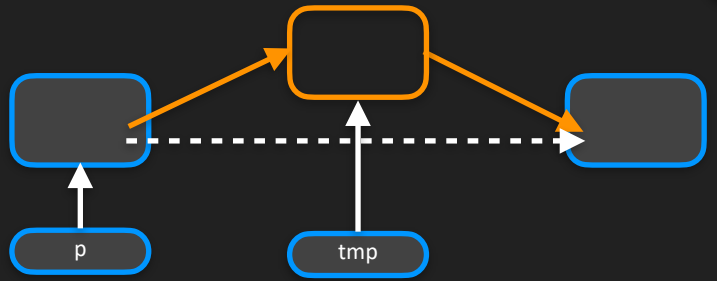
```
// to insert a new node with val x after the node p points to
p->next = new node{x,p->next};
if(p==tail) tail = p->next;
```



Inserting as the head needs slightly different code

```
// to remove p->next, without memory leak
node* tmp = p->next;
p->next = p->next->next;
if (tmp==tail) tail = p;
delete tmp;
```

Deleting the head needs slightly different code



Why Linked Lists

- O(1) insertion and deletion
  - Once we have the location where insertion/deletion happens
  - No existing data nodes are moved during insertion/deletion (except the one deleted)
- Insertion/deletion will not invalidate a node pointer already at hand (unless the node it is pointed to is the one deleted)
  - Can hold search results for a long time, with intervening insertions/deletions (of other nodes)

Why Arrays

- If only insertions at the end needed, arrays are better than lists
  - An important feature of arrays: random access. Directly access the i^{th} element in O(1) time
    - In a list, need to start from the beginning and sequentially proceed to the i^{th} element, in O(n) time
  - Arrays have less space overhead, faster sequential access (no pointer to follow), faster push_back (no memory allocation each time), and guaranteed to be contiguous in physical memory, which allows random access and also leads to better hardware performance ("cache-friendly")

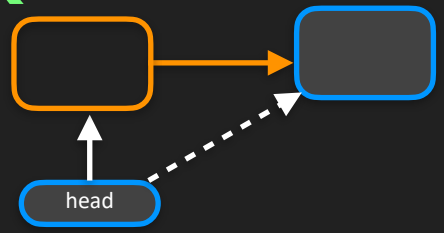
Packaging a List

- To work with a list (in particular to allow push_back and push_front), we need to maintain two pointers, to the front and the back of the list
- Why not make another struct which maintains these two pointers internally?
- E.g., `struct list { node* head; node* tail; /*functions*/};`
- push_back and push_front will automatically update tail/head pointers. User code doesn't have to maintain them (or any pointers)
- When a list object goes out of scope its destructor can deallocate all memory: `list::~list() { while (head) pop_front(); }`

pop_front and pop_back

- Popping from the front of a list:

```
void pop_front() {
  if (!head) return;
  node* tmp = head;
  head = head->next;
  if (tmp==tail) tail = nullptr;
  delete tmp;
}
```



Almost the same code as deleting p->next, but with p->next replaced with head.

- How about pop_back? We need the pointer to the node before tail (so that we can make tail point to it) without O(n) search.
  - Maintain one more pointer in struct list? But how do we update that then?
  - Idea: Every node maintains a pointer to the previous node as well.

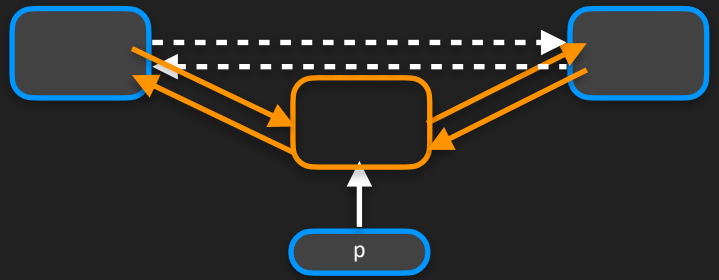
Doubly Linked List

- ```
  struct node {
    int val; node* next = nullptr; node* prev = nullptr;
  };
  ```

- Now, to delete a node, enough to have a pointer to that node itself

```
void list::erase(node* p) { // assumes p is a non-null pointer
  if(p==head) head=p->next; else p->prev->next = p->next;
  if(p==tail) tail=p->prev; else p->next->prev = p->prev;
  delete p;
}
```

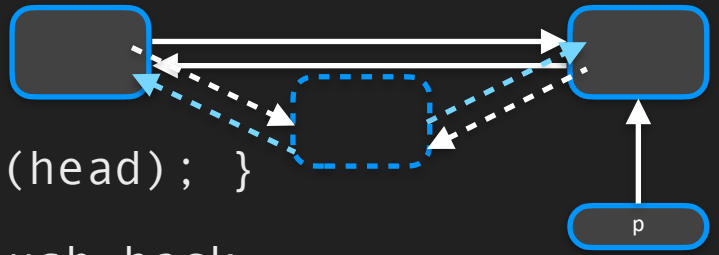
- ```
  void list::pop_front() { erase(head); }
  void list::pop_back()  { erase(tail); }
  ```



Doubly Linked List: Insert

- `void list::insert_before(node* p, int x) { // p non-null`

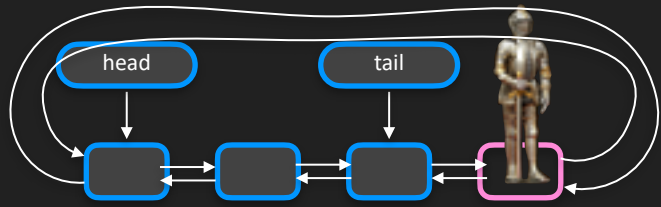
```
  node* tmp = new node{x,p,p->prev};
  if(p==head) head=tmp; else p->prev->next = tmp;
  p->prev = tmp;
}
```



- `void list::push_front() { insert_before(head); }`
- Can symmetrically define `insert_after` and `push_back`
  - Or can do `insert_before(p->next, x)` unless p is `nullptr`
  - Cannot use `insert_before` for `push_back`
- Can avoid special cases using a "sentinel" node (like in std::list)

Sentinel Node

- The code for list operations become cleaner/more uniform if we use a sentinel "end" node, beyond the tail
- The prev field of sentinel will point to the tail. No separate tail pointer.
  - Avoids separate check for inserting at tail (which is now just a standard insert before the sentinel)
- Sentinel's next field can store the head. No separate head pointer.
- head's prev can point to sentinel: avoids special cases for head
- Keep just the sentinel node itself in the list




```
struct list { node sentinel; /*functions*/};
```